JWT 게시판 구현 보고서

오늘의 맛집 후기

1 서비스 소개와 구현 범위

학번: 202410034　이름: 승연　제출일: 2026-10-26

서비스 이름: 오늘의 맛집 후기

기술 조합과 버전: Vue 3.5.43, Vite 8.3.4 / Node.js Express 5.3.0 / Node.js v24.18.0, jsonwebtoken 9.0.3, sql.js 1.14.2, SQLite 파일 DB

GitHub 저장소 URL: https://github.com/nguyenthingocduyen0405/CRUD

소스 코드 기준: main 브랜치. 최종 제출 기준 커밋 SHA 는 LMS 에 제출한다.

이 서비스는 다녀온 맛집과 음식을 기록하고 공유하는 게시판이다. 대상 사용자는 학교 주변이나 여행지에서 먹을 곳을 찾는 학생들이다. 후기는 로그인 없이 누구나 읽을 수 있고, 글 작성은 로그인한 사용자만 할 수 있으며, 수정과 삭제는 작성자 본인만 가능하도록 서버에서 검사한다. 기술 조합은 수업에서 Express 와 Vue 실습을 진행해 구조가 익숙했기 때문에 Vue 와 Node.js Express 를 선택했다. 제공 템플릿의 CRUD, 저장소, 화면 구조는 유지했고 인증 관련 TODO 와 화면 안내 문구만 변경했다.

기능	완료 여부	관련 파일	비고 또는 제한
공개 목록과 상세	완료	backend/src/app.js	토큰 없이 200 조회
로그인과 JWT 발급	완료	backend/src/auth.js issueToken	HS256, TTL 90 초
인증과 내 정보 조회	완료	backend/src/auth.js currentUserId	요청마다 검증
본인 글 작성, 수정, 삭제	완료	backend/src/auth.js assertOwner	타인 글은 403
만료, 로그아웃, 오류 표시	완료	frontend/src/api.js, App.vue	401 시 상태 해제
DB 데이터 유지	완료	backend/src/store.js	data/board.sqlite

2 구조와 변경 내역

전체 구조는 브라우저(127.0.0.1:4173 빌드 화면), Vite preview 서버의 /api 프록시, Express API 서버(127.0.0.1:8000), sql.js 기반 SQLite 파일(backend/data/board.sqlite) 순서로 연결된다.

로그인 요청 흐름: 로그인 폼에서 email 과 password 를 입력하면 frontend/src/api.js 의 login 함수가 POST /api/auth/login 으로 JSON 을 보낸다. 서버는 backend/src/users.js 의 verifyCredentials 로

계정을 확인하고, 성공하면 backend/src/auth.js 의 issueToken 이 JWT 를 발급한다. 프런트는 응답의 accessToken 과 user 를 자바스크립트 메모리 변수 session 에만 저장한다. localStorage 와 sessionStorage 는 사용하지 않으므로 새로고침하면 로그인 상태가 해제된다.

게시글 수정 요청 흐름: 수정 저장을 누르면 api.js 의 request 함수가 PUT /api/posts/{id}를 보내면서 authorizationHeaders 가 메모리의 토큰으로 Authorization: Bearer 헤더를 붙인다. 서버 라우트는 currentUserId 로 토큰을 검증해 사용자 ID 를 얻고, 글 존재를 확인한 뒤 assertOwner 로 작성자와 비교하고, 입력을 검사한 다음에만 DB 를 변경한다. 토큰은 /api/me 와 글 등록, 수정, 삭제 요청에만 전송되고 공개 목록과 상세 조회에는 전송되지 않는다.

변경한 파일과 함수	**제공 코드의 원래 역할**	**내가 변경한 내용**	**변경 이유**
backend/src/auth.js issueToken	501 NOT_IMPLEMENTED 반환	HS256 과 서버 키로 sub, exp, iss, aud 를 담은 JWT 발급	BE-01 과제 요구사항
backend/src/auth.js secretKey(모듈 상수)	없음	JWT_SECRET 이 64 자리 16 진수가 아니면 서버 시작을 거부하고 32 바이트 키로 변환	예시 키와 형식 오류 거부
backend/src/auth.js currentUserId	데모 ID 1 을 고정 반환	Bearer 헤더 추출, 서명과 만료와 클레임 검증, 계정 존재 확인 후 ID 반환	BE-02 과제 요구사항
backend/src/auth.js assertOwner	빈 함수	로그인 사용자와 글의 authorId 가 다르면 403 FORBIDDEN	BE-03 과제 요구사항
frontend/src/api.js login	오류만 던짐	로그인 API 호출 후 토큰과 사용자를 메모리에 저장	FE-01
frontend/src/api.js authorizationHeaders	빈 객체 반환	토큰이 있을 때만 Bearer 헤더 생성	FE-02
frontend/src/api.js request	401 처리 없음	401 응답이면 메모리 세션 해제	FE-03
frontend/src/api.js canEdit	항상 true	로그인 사용자 ID 와 글의 authorId 비교	FE-04
frontend/src/App.vue	데모 배너, 단일 오류 문구	배너를 서비스 안내로 교체, 응답 code 별 안내, 비로그인 작성 안내, 404 시 목록 갱신	FE-03, FE-04

변경한 파일과 함수	제공 코드의 원래 역할	내가 변경한 내용	변경 이유
backend/src/app.js 시작 로그	인증 미완성 안내 출력	서비스 이름으로 로그 문구 변경	최종본 정리

작성 출처 구분: 제공 코드(라우트, 저장소, 계정 검증, 화면 골격)는 그대로 유지했다. 위 표의 변경 내용은 AI 도구(Claude Code)가 초안을 작성했고, 본인이 코드를 읽고 테스트로 검증했다. 상세 내역은 5 장에 기록한다. 보조 파일로 backend/.env(새 비밀 키, 저장소에는 포함하지 않음)와 docs/evidence/api-test-log.txt(테스트 기록)를 추가했다.

3 핵심 코드 설명

3.1 로그인과 JWT 발급

파일 / 함수: backend/src/app.js 로그인 라우트, backend/src/auth.js issueToken

```
app.post("/api/auth/login", (req, res) => {
  const input = fields(req.body, { email: 254, password: 128 });
  const user = verifyCredentials(input.email, input.password);
  if (!user) throw new ApiError(401, "INVALID_CREDENTIALS", "...");
  const accessToken = issueToken(user);
  res.json({ accessToken, expiresIn: Number(process.env.TOKEN_TTL || 90), user });
});

export function issueToken(user) {
  return jwt.sign({}, secretKey, {
    algorithm: "HS256",
    subject: String(user.id),
    expiresIn: Number(process.env.TOKEN_TTL || 90),
    issuer: ISSUER,
    audience: AUDIENCE,
  });
}
```

입력: 로그인 요청 본문의 email 과 password 다. fields 함수가 두 필드만 허용하고 문자열 여부와 길이를 검사하며, 형식이 틀리면 400 BAD_INPUT 을 반환한다. 처리: 제공된 verifyCredentials 가 scrypt 해시와 timingSafeEqual 로 비밀번호를 비교한다. 계정 확인에 실패하면 토큰을 발급하지 않고 401 INVALID_CREDENTIALS 를 반환하므로, 임의의 이메일로 사용자가 만들어지지 않는다. 확인된 사용자에게만 issueToken 이 HS256 알고리즘과 서버 비밀 키로 JWT 를 서명한다. 토큰에는 문자열 사용자 ID 인 sub, 발급 후 90 초(TOKEN_TTL)의 만료 시각 exp, 발급자 iss(webservice-class), 사용 대상 aud(board-api)가 들어가며 비밀번호나 해시는 넣지 않는다. 출력: 200 과 accessToken, expiresIn, user(id, name, email)다. 비밀 키는 .env 의 JWT_SECRET 에서 읽는데, auth.js 모듈을 읽을

때 64 자리 16 진수인지 검사해 32 바이트 Buffer 로 변환하고, 빈 값이나 형식 오류나 제공 예시 키(16 진수가 아님)는 서버 시작 단계에서 거부한다.

3.2 브라우저 토큰 저장과 전송

파일 / 함수: frontend/src/api.js 의 login, authorizationHeaders, request, clearSession

```
let session = { accessToken: "", user: null };

export async function login(email, password) {
  const data = await request("/auth/login", {
    method: "POST",
    body: JSON.stringify({ email, password }),
  });
  session = { accessToken: data.accessToken, user: data.user };
  return data.user;
}

function authorizationHeaders() {
  return session.accessToken
    ? { Authorization: `Bearer ${session.accessToken}` }
    : {};
}
```

입력: 로그인 폼의 email 과 password 다. 처리: login 이 POST /api/auth/login 의 성공 응답에서 accessToken 과 user 를 모듈 변수 session 에 저장한다. 저장 위치는 자바스크립트 메모리뿐이고 localStorage 와 sessionStorage 에는 저장하지 않으므로, 새로고침이나 탭 종료 시 로그인 상태가 해제되는 것이 의도된 동작이다. 이후 모든 API 호출은 request 를 거치는데, authorizationHeaders 가 토큰이 있을 때만 Authorization: Bearer 헤더를 만들고 토큰이 없으면 빈 객체를 반환해 Bearer undefined 같은 잘못된 헤더를 만들지 않는다. 로그아웃 버튼은 clearSession 으로 메모리를 비우고 화면의 로그인 표시와 수정, 삭제 버튼을 즉시 갱신한다. 실패 조건: 로그인 실패 응답(401 INVALID_CREDENTIALS)이면 session 에 아무것도 저장하지 않고 오류 문구를 표시한다. request 는 401 응답을 받으면 clearSession 을 호출해 화면 로그인 상태도 함께 해제한다.

3.3 서버 JWT 검증

파일 / 함수: backend/src/auth.js currentUserId

```
export function currentUserId(req) {
  const match = /^Bearer₩s+(₩S+)$/.exec(req.get("Authorization") || "");
  if (!match)
    throw new ApiError(401, "TOKEN_REQUIRED", "로그인 토큰이 필요합니다.");
```

```
  let payload;
  try {
    payload = jwt.verify(match[1], secretKey, {
      algorithms: ["HS256"], issuer: ISSUER, audience: AUDIENCE,
    });
  } catch (error) {
    if (error instanceof jwt.TokenExpiredError)
      throw new ApiError(401, "TOKEN_EXPIRED", "...");
    throw new ApiError(401, "INVALID_TOKEN", "...");
  }
  if (typeof payload.exp !== "number" || typeof payload.sub !== "string" ||
      !/^[1-9]₩d*$/.test(payload.sub))
    throw new ApiError(401, "INVALID_TOKEN", "...");
  const userId = Number(payload.sub);
  if (!findPublic(userId))
    throw new ApiError(401, "INVALID_TOKEN", "...");
  return userId;
}
```

입력: 보호 API 요청의 Authorization 헤더다. 처리: 헤더가 없거나 Bearer 형식이 아니면 401 TOKEN_REQUIRED 다. 토큰이 있으면 jwt.verify 가 허용 알고리즘(HS256 고정), 서명, 만료, iss, aud 를 한꺼번에 검증한다. 내용을 읽기만 하는 decode 는 서명을 확인하지 않기 때문에 사용하지 않았다. 정상 서명이지만 만료된 토큰은 TokenExpiredError 로 구분되어 401 TOKEN_EXPIRED 가 되고, 서명 변조나 클레임 불일치는 401 INVALID_TOKEN 이 된다. jwt.verify 는 exp 가 아예 없는 토큰을 통과시키기 때문에 exp 의 존재와 sub 가 양의 정수 형태의 문자열인지를 추가로 직접 검사한다. 마지막으로 sub 가 가리키는 계정이 실제로 존재하는지 findPublic 으로 확인하고, 이렇게 검증을 통과한 사용자 ID 만 반환해 /api/me 와 글 등록, 수정, 삭제 라우트에 전달한다. 요청 본문에 들어온 사용자 정보는 신뢰하지 않는다. 출력: 검증된 사용자 ID(숫자)다. 모든 401 응답에는 공통 오류 처리기가 WWW-Authenticate: Bearer 헤더를 붙인다.

3.4 작성자 권한 검사

파일 / 함수: backend/src/auth.js assertOwner, backend/src/app.js 수정 라우트

```
export function assertOwner(userId, authorId) {
  if (userId !== authorId)
    throw new ApiError(403, "FORBIDDEN",
      "본인이 작성한 글만 수정하거나 삭제할 수 있습니다.");
}

app.put("/api/posts/:id", (req, res) => {
  const userId = currentUserId(req),
```

```
    id = postId(req),
    p = getPost(id);
  assertOwner(userId, p.authorId);
  res.json(store.update(id, fields(req.body, { title: 100, content: 5000 })));
});
```

비교 대상: 토큰 검증을 통과한 현재 사용자 ID 와 DB 에 저장된 글의 authorId 다. 글을 등록할 때 서버가 작성자를 검증된 사용자 ID 로 저장하므로(요청 본문의 authorId 같은 추가 필드는 fields 에서 400 BAD_INPUT 으로 거부), 다른 사람 이름으로 글을 만들 수 없다. 수정과 삭제 라우트는 인증(currentUserId), 글 존재 확인(getPost, 없으면 404), 작성자 비교(assertOwner), 입력 검증(fields) 순서로 처리하고 모든 검사를 통과했을 때만 DB 를 변경한다. 401 과 403 의 차이: 401 은 토큰이 없거나 무효해서 누구인지 확인하지 못한 경우이고, 403 은 사용자는 확인되었지만 다른 사람의 글이라 권한이 없는 경우다. 수정에 성공해도 store.update 는 title, content, updatedAt 만 바꾸므로 기존 작성자와 createdAt 은 유지된다. 프런트에서 canEdit 로 타인 글의 버튼을 숨기지만, 버튼과 무관하게 직접 API 를 호출해도 이 서버 검사가 거부한다는 것을 T07 과 T08 로 확인했다.

4 테스트 결과와 증거

실행 환경: Windows 11, Node.js v24.18.0, Python 3.13.15(검사 스크립트 실행), 브라우저 Chromium(Playwright 자동 브라우저). 실행 명령은 backend 폴더에서 npm ci 와 npm start, frontend 폴더에서 npm ci 와 npm run build 와 npm run preview 다.

로컬 배포 확인: frontend 에서 npm run build 가 성공해 dist 가 생성되었고, npm run preview 로 http://127.0.0.1:4173 접속 시 페이지가 200 으로 열리며 /api/posts 프록시 요청도 200 으로 백엔드에 연결됨을 확인했다.

테스트 일시 및 코드 버전: API 검사는 2026-10-24, 화면 검사는 2026-10-25(모두 UTC, docs/evidence/api-test-log.txt 에 기록)에 제출 코드와 동일한 버전으로 수행했다.

토큰 TTL: 일반 테스트 90 초, 만료 테스트(T10)는 TOKEN_TTL=3 으로 서버를 재시작해 수행했다.

아래 표의 실제 결과는 Python urllib 스크립트로 서버에 직접 요청해 관찰한 HTTP 상태와 JSON code 이며, 전체 기록은 docs/evidence/api-test-log.txt 에 있다. 화면 표시가 포함된 항목은 4173 번의 빌드 화면을 Playwright 자동 브라우저(Chromium)로 조작해 확인했고, 화면 캡처(01 번부터 07 번 png 파일)와 토큰을 가린 요청 기록(network-capture.txt)을 docs/evidence 에 포함했다.

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T01	토큰 없이 목록과 상세 조회	200 공개 조회	200 / 해당 없음(목록, 상세 모두)	PASS	로그 T01
T02	Alice 로그인	200, 토큰과 사용자	200, accessToken 과 user 반환, 화면에 Alice 님 로그인 중 표시	PASS	로그 T02, 02_alice_login.png
T03	틀린 비밀번호, 없는 이메일 로그인	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS(2 건)	PASS	로그 T03
T04	토큰 없이 /me 와 글 등록	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED(2 건)	PASS	로그 T04
T05	Alice 토큰으로 글 작성 후 /me	201, authorId 1, /me Alice	201(Location 포함), authorId 1, /me 200	PASS	로그 T05
T06	Alice 본인 글 수정과 삭제	200 과 204, DB 반영	200(createdAt 유지), 204(빈 본문), 재조회 404	PASS	로그 T06
T07	Bob 토큰으로 Alice 글 수정	403, 원문 유지	403 / FORBIDDEN, 재조회 결과 변경 없음	PASS	로그 T07
T08	Bob 토큰으로 Alice 글 삭제	403, 글 유지	403 / FORBIDDEN, 글 존재 유지	PASS	로그 T08
T09	서명 변조 토큰으로 /me 와 변경 요청	401 INVALID_TOKEN	401 / INVALID_TOKEN(GET, POST, PUT, DELETE), 데이터 변경 없음	PASS	로그 T09
T10	TTL 3 초로 발급 후 만료 뒤 요청	401 TOKEN_EXPIRED, 화면 해제	401 / TOKEN_EXPIRED(4 종 요청), 데이터 변경 없음, 화면 재로그인 안내와 로그인 해제	PASS	로그 T10, 07_token_expired.png

ID	수행 절차와 사용자	예상 결과	실제 HTTP 와 code	판정	증거
T11	로그아웃 후 /me 요청	헤더 없는 요청, 401 TOKEN_REQUIRED	로그아웃 즉시 화면 해제, 토큰 없는 요청에 401 / TOKEN_REQUIRED	PASS	로그 T11, 06_logout.png
T12	Bob 등록 요청에 authorId, userId 추가	400, 위조 글 미생성	400 / BAD_INPUT(2 건), 글 수 변화 없음	PASS	로그 T12
T13	빈 제목, 공백 본문, 길이 초과 입력	400 BAD_INPUT	400 / BAD_INPUT(5 건), 데이터 변경 없음	PASS	로그 T13
T14	없는 ID 조회, 수정, 삭제와 잘못된 ID	404 와 400	999999 는 404 / NOT_FOUND(3 건), abc 는 400 / BAD_INPUT	PASS	로그 T14
T15	글 생성 후 서버 종료, 재실행	글과 작성자 유지	재시작 후 200, 제목과 작성자와 createdAt 동일	PASS	로그 T15
T16	로그인 후 새로고침	글 유지, 로그인 해제	새로고침 후 글 4 건 그대로 유지, 화면은 로그아웃 상태로 전환	PASS	04_T16_refresh.png
T17	무토큰 변경 요청과 Bob 본인 작업	무토큰 401, 본인 작업 성공	무토큰 PUT, DELETE 401 / TOKEN_REQUIRED, Bob 등록 201, 수정 200, 삭제 204	PASS	로그 T17

HTTP 응답 확인 기록

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
정상 조회, 로그인, 수정	200	200(T01, T02, T05, T06, T17)	로그 해당 행
새 글 등록	201 및 Location	201, Location: /api/posts/{id} 확인	로그 T05
본인 글 삭제	204 및 빈 본문	204, 응답 본문 0 바이트 확인	로그 T06, T17

확인 항목	기대 응답	실제 HTTP 와 code	증거 위치
입력 오류, 추가 작성자 필드	400 BAD_INPUT	400 / BAD_INPUT	로그 T12, T13, T14
잘못된 계정	401 INVALID_CREDENTIALS	401 / INVALID_CREDENTIALS	로그 T03
토큰 없음, 헤더 형식 오류	401 TOKEN_REQUIRED	401 / TOKEN_REQUIRED	로그 T04, T11, T14, T17
변조, 클레임 오류	401 INVALID_TOKEN	401 / INVALID_TOKEN	로그 T09
정상 서명의 만료 토큰	401 TOKEN_EXPIRED	401 / TOKEN_EXPIRED	로그 T10
다른 작성자의 변경	403 FORBIDDEN	403 / FORBIDDEN	로그 T07, T08
없는 글	404 NOT_FOUND	404 / NOT_FOUND	로그 T06, T14

보호 API 401 의 WWW-Authenticate 확인: 토큰 없음(T04), 만료(T10) 응답에서 WWW-Authenticate: Bearer 헤더를 확인했고 로그에 기록되어 있다.

401 과 403 의 화면 차이: 401 을 받으면 api.js 의 request 가 메모리 세션을 지우고 화면이 로그아웃 상태로 바뀌며 재로그인 안내 문구를 표시한다. 만료 토큰으로 요청한 직후의 화면이 07_token_expired.png 다. 403 은 로그인 상태를 유지한 채 본인이 작성한 글만 수정하거나 삭제할 수 있다는 안내만 표시한다. Bob 으로 로그인해 Alice 의 글을 열면 수정과 삭제 버튼이 보이지 않는 화면이 05_bob_view_alice_post.png 다.

실패한 변경 요청 후 데이터 불변 확인: T07, T08, T09, T10, T13 의 실패 응답 직후 공개 조회로 글을 다시 받아 원문과 비교했고 모두 변경되지 않았다(로그의 data unchanged 행).

여러 조건이 겹친 요청: 토큰 없이 없는 글(999999) 수정은 401 TOKEN_REQUIRED, Alice 의 정상 토큰으로 같은 글 수정은 404 NOT_FOUND 였다(로그 T14). Bob 이 Alice 글에 공백 제목을 보내면 입력 검증보다 권한 검사가 먼저 적용되어 403 FORBIDDEN 이었다(로그 EXTRA 행). Alice 가 자기 글에 같은 입력을 보내면 400 BAD_INPUT 이었다(로그 T13).

증거 1: 로그인 성공과 토큰이 첨부된 요청은 로그 T02 와 T05 에 있으며 토큰은 앞뒤 일부만 남기고 가렸다. 로그인 성공 화면은 02_alice_login.png, 빌드 화면에서 보낸 실제 요청의 Authorization 헤더 기록(토큰 가림)은 network-capture.txt 에 있다.

증거 2: Bob 이 Alice 의 글 수정과 삭제를 시도한 서버 응답은 로그 T07 과 T08 에 있다. Bob 에게 수정과 삭제 버튼이 보이지 않는 화면은 05_bob_view_alice_post.png 에 있다.

증거 3: 토큰 변조와 만료 시 서버 응답은 로그 T09 와 T10 에 있다. 만료 시 화면 처리는 07_token_expired.png 에 있다.

자동 검사: 저장소 루트에서 py tools/check_api.py --mode final 을 실행해 40 개 항목이 모두 통과했고, TOKEN_TTL=3 으로 서버를 재시작한 뒤 py tools/check_api.py --mode final --expiry 로 만료 포함 48 개 항목이 모두 통과했다. 이 도구는 수업에서 제공된 것으로 API 응답의 상태와 code 와 헤더를 자동 확인하며, 화면과 서버 재시작 검사는 별도로 수행했다.

5 AI 활용과 참고 자료

AI 사용 여부: 사용

전체 사용 범위: Anthropic Claude Code 를 사용해 시작 템플릿의 인증 TODO 구현(BE-01 부터 BE-03, FE-01 부터 FE-04), 테스트 수행과 기록, README 와 본 보고서 초안 작성을 진행했다. 제공 템플릿의 CRUD 와 저장소 코드는 AI 가 수정하지 않고 유지했다.

도구와 모델	**사용 목적**	**반영한 파일 또는 보고서 항목**	**반영 방식**
Claude Code(Claude Fable 5)	코드 생성	backend/src/auth.js, frontend/src/api.js, frontend/src/App.vue	생성 후 테스트로 검증해 반영
Claude Code(Claude Fable 5)	테스트	docs/evidence/api-test-log.txt, 자동 검사 실행	스크립트 작성과 실행
Claude Code(Claude Fable 5)	문장 정리	README.md, 본 보고서 초안	초안 작성 후 본인 확인

대표 사례 1: 백엔드 인증 구현

도구 / 날짜: Claude Code / 2026-10-25

요청한 내용: 과제 공지와 API 명세에 따라 backend/src/auth.js 의 issueToken, currentUserId, assertOwner 를 구현해 달라고 요청했다.

AI 가 제안한 내용: jsonwebtoken 라이브러리로 HS256 발급과 검증을 구현하고, 서버 시작 시 JWT_SECRET 형식을 검사하며, exp 존재와 sub 형식과 계정 존재를 추가로 확인하는 코드를 제안했다.

실제 반영 위치: backend/src/auth.js 전체

내가 확인하고 수정한 내용: auth.js 를 한 줄씩 읽으며 jwt.verify 의 허용 알고리즘이 HS256 하나로 고정되어 있는지, 토큰 payload 에 비밀번호나 해시 같은 민감 정보가 들어가지 않는지, 만료와 변조가 TOKEN_EXPIRED 와 INVALID_TOKEN 으로 구분되어 응답되는지 확인했다. 처음에는 exp 존재와 sub 형식을 추가로 검사하는 부분이 중복이라고 생각했지만, jwt.verify 가 exp 가 없는 토큰을 그대로 통과시킨다는 내용을 node-jsonwebtoken 문서에서 확인한 뒤 그대로 유지했다. 오류 code 와 문구가 docs/02_API 명세.md 와 일치하는지 대조했고, 제안 코드에서 고칠 오류를 찾지 못해 수정 없이 반영했다.

검증 방법과 결과: 수업 제공 자동 검사(final 40 개, expiry 포함 48 개) 전부 통과, T01 부터 T17 수동 시나리오의 API 요청 결과가 명세와 일치함을 docs/evidence/api-test-log.txt 로 확인했다.

대표 사례 2: 프런트 로그인 연결

도구 / 날짜: Claude Code / 2026-10-25

요청한 내용: FE-01 부터 FE-04 에 따라 로그인 연결, 토큰 전송, 응답 코드별 화면 처리, 작성자별 버튼 표시를 구현해 달라고 요청했다.

AI 가 제안한 내용: 토큰을 메모리 변수에만 저장하는 login 함수, 토큰이 있을 때만 Bearer 헤더를 만드는 authorizationHeaders, 401 시 세션을 지우는 request 처리, canEdit 비교, App.vue 의 code 별 안내 문구를 제안했다.

실제 반영 위치: frontend/src/api.js, frontend/src/App.vue

내가 확인하고 수정한 내용: api.js 와 App.vue 를 읽고 토큰이 localStorage 나 sessionStorage 가 아니라 메모리 변수 session 에만 저장되는지, authorizationHeaders 가 토큰이 없을 때 빈 객체를 반환해 Bearer undefined 같은 잘못된 헤더가 생기지 않는지, 401 응답에서 clearSession 이 호출되어 화면이 로그아웃 상태로 바뀌는지 확인했다. 브라우저에서 로그인, 새로고침(로그인 해제), 로그아웃, Bob 으로 Alice 글을 열었을 때 수정·삭제 버튼이 숨겨지는 것을 직접 조작해 확인했고, 동작이 명세와 일치해 수정 없이 반영했다.

검증 방법과 결과: npm run build 성공, 4173 번 빌드 화면과 API 프록시 연결 확인, T04 와 T10 과 T11 의 API 결과 확인. 화면 동작은 Playwright 자동 브라우저로 빌드 화면을 조작해 확인했고 캡처를 docs/evidence 에 포함했다.

그 밖의 사용 내역 요약: 테스트 스크립트 작성과 실행, 서버 재시작을 포함한 T15 검증, Playwright 자동 브라우저를 이용한 화면 확인과 캡처, README.md 초안, 본 보고서 초안 작성에 사용했다.

참고 자료

자료 제목과 URL	사용한 위치	반영 내용 및 출처 표시
수업 제공 시작 템플릿과 공통 API 명세(docs/02_API 명세.md)	프로젝트 전체	제공 코드 유지, 명세에 맞춰 구현
node-jsonwebtoken, https://github.com/auth0/node-jsonwebtoken	backend/src/auth.js	sign 과 verify 옵션 사용법 참고
Vite 공식 문서, https://vite.dev/guide/	frontend 빌드와 프록시	빌드와 preview 명령 참고

6 이해도 확인과 회고

1. 요청 본문의 authorId 를 믿으면 어떤 문제가 생기며, 내 코드는 어떻게 막는가?

   요청 본문의 authorId 를 그대로 저장하면 누구나 다른 사람 이름으로 글을 만들거나, 타인 글의 작성자를 바꿔 수정 권한을 얻는 작성자 위조가 가능해진다. 내 코드는 두 겹으로 막는다. 첫째, app.js 의 fields 함수가 등록과 수정 본문에 title 과 content 두 필드만 허용하므로 authorId 나 userId 같은 추가 필드가 오면 400 BAD_INPUT 으로 거부한다(T12 에서 확인). 둘째, 글을 저장할 때 작성자는 본문이 아니라 currentUserId 가 토큰을 검증해 얻은 사용자 ID 로 서버가 결정한다.

2. 수정 버튼을 숨기는 것만으로 충분하지 않은 이유는 무엇이며, 서버의 어떤 함수가 권한을 검사하는가?

   버튼은 화면 표시일 뿐이고, curl 이나 API 클라이언트로 서버에 직접 PUT 이나 DELETE 요청을 보내면 버튼과 무관하게 요청이 도착한다. 따라서 서버가 요청마다 검사해야 한다. 내 코드에서는 backend/src/auth.js 의 assertOwner 가 토큰 검증으로 얻은 사용자 ID 와 DB 에 저장된 글의 authorId 를 비교해 다르면 403 FORBIDDEN 을 던지고 DB 를 변경하지 않는다. T07 과 T08 에서 Bob 의 정상 토큰으로 Alice 글을 직접 요청해 403 과 데이터 불변을 확인했다.

3. JWT decode 와 verify 의 차이는 무엇이며, 내 코드는 어떤 검증을 수행하는가?

   decode 는 토큰의 내용을 base64 로 풀어 읽기만 하고 서명을 확인하지 않으므로, 누구나 만들거나 고친 가짜 토큰도 통과한다. verify 는 서버 비밀 키로 서명을 다시 계산해 위조 여부를 확인하고 만료와 클레임도 함께 검사한다. 내 코드는 jwt.verify 에 허용 알고리즘을 HS256 하나로 고정해 서명, exp 만료, iss, aud 를 검증하고, verify 가 보장하지 않는 exp 의 존재와 sub 의 문자열 양의 정수 형식을 추가로 직접 검사하며, 마지막으로 sub 의 계정이 실제로 존재하는지 확인한다. 서명 변조는 T09 에서 401 INVALID_TOKEN, 만료는 T10 에서 401 TOKEN_EXPIRED 로 구분됨을 확인했다.

4. 로그아웃, 토큰 만료, 새로고침은 각각 무엇을 바꾸는가? 이미 복사된 토큰은 어떻게 되는가?

로그아웃은 이 탭의 자바스크립트 메모리에 있는 토큰과 사용자 상태를 지워 화면을 비로그인으로 바꾸지만 서버에는 아무 변화가 없다. 토큰 만료는 서버 쪽 검증 결과가 바뀌는 것으로, exp 가 지난 토큰은 어떤 요청에서도 401 TOKEN_EXPIRED 로 거부된다. 새로고침은 메모리가 초기화되어 로그인 상태만 해제되고, SQLite 파일에 저장된 게시글 데이터는 유지된다. 이미 복사된 토큰은 로그아웃해도 무효화되지 않으며 exp 시각까지는 서버 검증을 통과한다. 이 과제의 로그아웃은 서버 측 무효화 기능이 아니고, 짧은 TTL(90 초)이 유출 피해를 줄이는 역할을 한다.

5. 가장 어려웠던 오류 한 가지와 원인 확인, 수정, 재검증 과정을 설명하라.

가장 어려웠던 오류는 백엔드 인증을 구현한 직후 서버가 시작조차 되지 않은 문제였다. backend 에서 npm start 를 실행하자 "JWT_SECRET 은 64 자리 16 진수 문자열이어야 합니다"라는 오류와 함께 프로세스가 바로 종료되었다. 원인 확인: 오류 문구로 auth.js 의 secretKey 시작 검사에서 발생한 것을 찾았고, backend/.env 를 열어 보니 .env.example 을 복사한 그대로여서 16 진수 형식이 아닌 예시 키가 남아 있었다. 수정: backend/README.md 의 안내대로 node -e "console.log(require('node:crypto').randomBytes(32).toString('hex'))"로 새 64 자리 16 진수 키를 생성해 .env 의 JWT_SECRET 값을 교체했다(키 자체는 저장소와 보고서에 넣지 않았다). 재검증: 서버가 정상 시작되고 Alice 로그인(T02)이 200 으로 토큰을 반환하는 것, 자동 검사 48 개 항목이 모두 통과하는 것을 다시 확인했다. 이 과정에서 비밀 키 형식 검사를 서버 시작 시점에 두면 잘못된 키로 서비스가 조용히 동작하는 사고를 막을 수 있다는 점을 배웠다.

현재 남은 문제와 다음에 개선할 점: 기능상 알려진 문제는 없다. 화면 동작은 자동 브라우저로 확인했으므로 제출 전에 본인 브라우저로 한 번 더 직접 확인한다.

제출 확인: [v] 실제 코드와 설명 일치 [v] 테스트 직접 수행 [v] AI 활용 및 출처 기록 [v] 비밀 키와 토큰 제거

GitHub 확인: [v] 루트 README 작성 [v] 보고서 PDF 와 증거 업로드 [v] README 문서 링크 확인 [v] 교수자 접근 가능 [ ] 저장소 URL 과 커밋 SHA 제출